

Research Task: Terraform Workspaces (Task 15)



Zaeem Attique Ashar
Cloud Intern

Task Description:

Terraform workspaces is a feature of terraform that helps us manage different environments and infrastructures. This research report covers the creation and lifecycle management of workspaces as well as operations such as listing, selecting, and showing workspaces.

Task15.1: Terraform Workspaces and Their Purpose	2
Task15.2: Terraform Workspace Commands	3
Task15.3: Workspace Specific Variables and State Files	3
Task15.4: Applying Infrastructure Independently	5
Task15.5: Workspaces with Remote Backend	5
Task15.6: Advantages and Disadvantages of Workspaces	6
Task15.7: Troubleshooting	6

Task15.1: Terraform Workspaces and Their Purpose

Terraform Workspaces is a feature of terraform that we can use to manage different workspaces such as Production, Development, Staging, etc without affecting the other workspaces in the process. This is done by terraform as it makes a whole directory that stores the different state files of different workspaces. We can move between workspaces and the changes made in the workspace will remain local to it.

```
$ terraform workspace list
default
dev
* test
```

Task15.2: Terraform Workspace Commands

- To create a new Workspace, we can use the following command:
~ terraform workspace new <name of workspace>
- To see all the workspaces currently available, use the following command:
~ terraform workspace list
- To switch between the listed workspaces, use the following command:
~ terraform workspace select <name of the workspace>
- To delete one of the listed workspaces, use the following command:
~ terraform workspace delete <name of the workspace>
- To select a workspace if available, otherwise create it, use the command:
~ terraform workspace select -or-create <name>
- To get the help screen of terraform workspaces, use the command:
~ terraform workspace help

Task15.3: Workspace Specific Variables and State Files

When we use workspaces, instead of storing a single state file called *terraform.tfstate*, it creates a directory named *terraform.tfstate.d* which is then used to store different state files for different workspaces.

```
.terraform/  
└─ terraform.tfstate.d/  
   └─ dev/terraform.tfstate  
   └─ prod/terraform.tfstate
```

Terraform does not automatically load a different set of variables to form the tfvars file for different workspaces. They must be configured in either of the two methods listed below:

1. Use Variable Maps:

This method defines a variable block that contains different values mapped to different environments. We can then mention the environment and fetch the exact value specified in the variable block.

```
variable "instance_type" {  
  type = map(string)  
  default = {  
    dev = "t3.micro"  
    prod = "t3.large"  
  }  
}  
  
resource "aws_instance" "app" {  
  instance_type = var.instance_type[terraform.workspace]  
}
```

2. Workspace based Locals

This method configures the locals block inside the terraform code which specifies the value of the variable according to the environment/workspace. We can use the name of the workspace with a special expression to retrieve the value.

```
locals {
  env_config = {
    dev = {
      instance_type = "t3.micro"
    }
    prod = {
      instance_type = "t3.large"
    }
  }
}

resource "aws_instance" "app" {
  instance_type = local.env_config[terraform.workspace].instance_type
}
```

Task15.4: Applying Infrastructure Independently

As mentioned before, Terraform handles multiple state files for multiple workspaces automatically. But there is a need to make sure to check out the currently active workspace as Terraform will use the state file associated with the workspace, and the infrastructure changes will be made specifically.

```
terraform workspace new dev
terraform apply

terraform workspace new prod
terraform apply
```

Task15.5: Workspaces with Remote Backend

When using a remote backend combination such as Amazon S3 bucket paired with a DynamoDB Table, terraform only needs one of each to automatically store different state files as well as state locks. Terraform uses keys to store and fetch the state files of different workspaces and uses different entries in the table to store the lock. This reduces the complexity and redundancy of resources and allows users to apply changes to different workspaces simultaneously.

The state file keys stored in Amazon S3:

```
app/env:/dev/terraform.tfstate  
app/env:/prod/terraform.tfstate
```

Task15.6: Advantages and Disadvantages of Workspaces

Environment Isolation - Separate state files prevent accidental changes

Simplified Management - Single codebase for multiple environments

Cost Efficiency - Reuse S3 bucket and DynamoDB table

Parallel Operations - Multiple teams can work simultaneously

Easy Switching - Quick environment transitions with one command

Reduced Complexity - No need for separate backend configurations

Below are listed a few disadvantages of Terraform Workspaces:

Variable Management - No automatic variable file switching

Naming Conflicts - Must ensure workspace names don't conflict

State Access - All workspaces share backend access permissions

Not for Multi-Region - Better suited for environments, not geographic isolation

Backend Migration - Moving between backends requires careful planning

Task15.7: Troubleshooting

Issue: Cannot delete workspace

Solution: Destroy all resources first, then delete workspace

Issue: Wrong workspace applied

Solution: Always run terraform workspace show before operations

Issue: State file conflicts

Solution: Ensure only one person/process modifies a workspace at a time

Issue: Variables not loading

Solution: Implement workspace-specific variable maps or locals